

PRÁCTICA N.º 5 — ENUNCIADO

Comunicación de alto rendimiento con gRPC

El archivo .proto como contrato, llamadas unarias y de flujo, evolución sin romper clientes y una comparación medida contra REST

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 4 — Comunicación entre Microservicios
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 4 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica5-grpc

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Reconocer cuándo un servicio interno se comunica mejor con gRPC que con REST o con GraphQL, y saber justificarlo con una medida y no con una preferencia.
- Escribir un archivo .proto legible: mensajes, servicio, tipos y numeración de campos, entendiendo que el número es lo que viaja y el nombre no.
- Implementar un servidor gRPC en Node.js con llamadas unarias y probarlo sin escribir todavía un cliente.
- Escribir un cliente gRPC y manejar los errores con los códigos de estado propios de gRPC, distinguiéndolos de los códigos de HTTP.
- Implementar una llamada con flujo del servidor y demostrar en qué caso concreto conviene frente a una respuesta única.
- Evolucionar un contrato agregando un campo sin romper a los clientes que todavía usan la versión anterior.
- Provocar y explicar la ruptura silenciosa que se produce al reutilizar el número de un campo, y prevenirla con reserved.
- Medir el tamaño de la carga y el tiempo de una misma operación en REST y en gRPC, y sostener la conclusión con esos números.
- Empaquetar el servicio gRPC en un contenedor y consumirlo desde el servicio REST por nombre de servicio dentro de la red de Docker.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 4 — Comunicación entre Microservicios**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Archivo .proto:** el contrato escrito. Declara los mensajes y las operaciones, y es el único documento que cliente y servidor comparten; los dos lo leen, ninguno lo adivina.
- **message:** la estructura de datos que viaja. Cada campo lleva un tipo, un nombre y un número: el número es lo que se transmite, el nombre solo existe en el código fuente.
- **service:** el conjunto de operaciones que el servidor ofrece. Cada rpc declara qué mensaje recibe y qué mensaje devuelve.
- **Llamada unaria:** un mensaje de ida y un mensaje de vuelta. Es la forma más parecida a una petición HTTP corriente y con la que empezaremos.

- **Streaming:** una llamada en la que uno de los dos extremos, o los dos, envía una secuencia de mensajes por la misma conexión en lugar de una única respuesta.

Repaso: La justificación conceptual —por qué HTTP/2 permite multiplexar, qué gana un formato binario frente a uno de texto, en qué punto de una arquitectura conviene cada estilo de comunicación— está en las diapositivas del Tema 4. Esta práctica es la ejecución: aquí se escribe, se rompe a propósito y se mide.

3. Requisitos previos

Requisito	Verificación	Observación
Node.js 20 o superior	<code>node --version</code>	La versión 20 o la 22 LTS
npm 10+	<code>npm --version</code>	Viene con Node
Servicio REST de la Práctica 3	<code>curl http://localhost:3000/usuarios</code>	Se mide contra él en el Laboratorio 6 y lo consume por gRPC al final
Docker Desktop o Engine 24+	<code>docker --version</code>	Necesario solo para el Laboratorio 6
grpcurl 1.8 o superior	<code>grpcurl --version</code>	Se instala en el Laboratorio 0; permite llamar al servicio sin escribir cliente
Editor de código	VS Code recomendado	Extensión «vscode-proto3» para resaltar la sintaxis del .proto
Git	<code>git --version</code>	Para subir la entrega al repositorio

Atención: Cree una carpeta de trabajo llamada `practica5-grpc` y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo \$. NO escriba el símbolo \$ al copiar el comando.

Esta práctica NO reemplaza el servicio REST de la Práctica 3 ni la capa GraphQL de la Práctica 4: los tres conviven. gRPC se agrega para la comunicación entre servicios internos, y al final el REST seguirá siendo la puerta pública.

Varios laboratorios necesitan dos terminales abiertas a la vez: una con el servidor corriendo y otra para el cliente. No cierre el servidor entre laboratorios.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 19 capturas y se responden las 8 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guía	7 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica5-grpc: la carpeta proto/ con el contrato, proto-v1/ con la versión congelada, src/ con el servidor y los clientes, Dockerfile, docker-compose.yml y README.md con las instrucciones para levantar todo.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, la tabla de mediciones REST contra gRPC y las respuestas a las preguntas de reflexión.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Consejo: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — Entorno y proyecto base

En la Práctica 3 construyó un servicio REST y en la Práctica 4 una capa GraphQL sobre él. Los dos resuelven la comunicación con el exterior: un navegador, una aplicación móvil, un cliente que usted no controla. Hoy se agrega el tercer estilo, el que se usa hacia adentro: entre servicios que usted sí controla, que hablan muchas veces por segundo y que no necesitan ser legibles para un humano. Antes de escribir nada, confirme que las herramientas están donde deben estar.

Paso 1. Verifique las versiones y el servicio que ya tiene

Abra una terminal y ejecute los cuatro comandos. El último comprueba que el servicio REST de la Práctica 3 sigue funcionando: lo vamos a necesitar para medir y, al final, para que consuma el servicio gRPC.

```
$ node --version
$ npm --version
$ docker --version
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/usuarios
# el último comando debe responder 200: es el servicio REST de la Práctica 3
```

Si el último comando no responde 200, levante el servicio de la Práctica 3 antes de continuar. Si aún no lo tiene, el Laboratorio 6 indica el mínimo que necesita construir para poder medir.

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal, incluido el código 200 del servicio REST. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: Ya cuenta con un servicio REST y con una capa GraphQL. Describa una situación concreta de su proyecto integrador —quién llama a quién, cada cuánto y con qué volumen de datos— en la que agregar un tercer estilo de comunicación estaría justificado. Si no encuentra ninguna, dígalo y explique por qué. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 2. Cree el proyecto e instale las dos bibliotecas

gRPC en Node.js se resuelve con dos paquetes: uno implementa el transporte y el otro lee el contrato.

```
$ mkdir practica5-grpc && cd practica5-grpc
$ npm init -y
$ npm install @grpc/grpc-js @grpc/proto-loader
```

```
$ mkdir proto src informe
# grpc-js: el servidor, el cliente y el transporte sobre HTTP/2
# proto-loader: lee el .proto en tiempo de ejecución, sin generar código
```

En otros lenguajes el .proto se compila con protoc y genera clases. En Node.js con proto-loader el contrato se lee al arrancar el proceso: el resultado es el mismo, pero el archivo .proto tiene que viajar con la aplicación. Recuérdelo: en el Laboratorio 6 esa diferencia decide una línea del Dockerfile.

Captura N.º 2: salida de npm install mostrando los dos paquetes instalados y la estructura de carpetas creada. → *péguela en la Hoja de Evidencias.*

Paso 3. Instale la herramienta de prueba

A una API REST se le puede hablar con curl porque el mensaje es texto. A un servicio gRPC no: el mensaje es binario y hay que interpretarlo con el contrato. grpcurl hace exactamente eso, y permite probar el servidor antes de tener un cliente escrito.

```
# Linux o WSL
$ curl -sL https://github.com/fullstorydev/grpcurl/releases/download/\
v1.9.1/grpcurl_1.9.1_linux_x86_64.tar.gz | tar -xz grpcurl
$ sudo mv grpcurl /usr/local/bin/ && grpcurl --version

# macOS con Homebrew
$ brew install grpcurl

# Windows con winget o Scoop
$ winget install fullstorydev.grpcurl
```

Si no puede instalarlo: Si su equipo no le permite instalar grpcurl, no se detenga: a partir del Laboratorio 3 usted mismo escribirá un cliente en Node.js que cumple la misma función. Indíquelo en la Hoja de Evidencias y reemplace las llamadas de grpcurl por ese cliente.

Con el entorno listo, lo primero no es el código del servidor: es el contrato. En gRPC el orden es siempre ese.

Laboratorio 1 — El archivo .proto como contrato

En REST el contrato es un documento OpenAPI que se escribe aparte y que nada obliga a cumplir. En GraphQL el esquema sí es obligatorio, pero se valida en el servidor. En gRPC el contrato es el punto de partida de los dos extremos: sin él no hay cliente ni servidor. Escribámoslo primero.

Paso 1. Escriba el contrato

Cree el archivo proto/estudiantes.proto. Es el servicio interno de expedientes académicos: guarda estudiantes y los devuelve por su cédula de identidad.

```
proto/estudiantes.proto
syntax = "proto3";
package estudiantes;

message Estudiante {
  string ci          = 1;
  string nombres     = 2;
  string apellidos   = 3;
  string carrera     = 4;
}

message EstudianteRequest { string ci = 1; }
message EstudianteResponse { Estudiante estudiante = 1; }
```

```
service EstudianteService {  
  rpc AgregarEstudiante (Estudiante) returns (EstudianteResponse);  
  rpc ObtenerEstudiante (EstudianteRequest) returns (EstudianteResponse);  
}
```

Léalo como un contrato y no como código: package define el espacio de nombres, cada message es una estructura que puede viajar, y service enumera las operaciones. Observe que ObtenerEstudiante no recibe una cadena suelta sino un mensaje propio: así, el día que la búsqueda necesite un segundo criterio, se agrega un campo sin cambiar la firma.

Por qué un mensaje de petición y no un tipo simple: proto3 no permite parámetros sueltos en un rpc: siempre entra un mensaje y siempre sale un mensaje. Esa restricción, que al principio parece burocracia, es lo que permite que el contrato crezca sin romperse.

Paso 2. Escriba el módulo que carga el contrato

El servidor y todos los clientes van a necesitar lo mismo: leer el .proto y construir el paquete. Escríbalo una sola vez en src/carga.js. La carpeta del contrato se lee de una variable de entorno, y en el Laboratorio 5 verá para qué sirve esa decisión.

```
src/carga.js  
const grpc = require("@grpc/grpc-js");  
const protoLoader = require("@grpc/proto-loader");  
const CARPETA = process.env.PROTO_DIR || __dirname + "/../proto";  
const definicion = protoLoader.loadSync(CARPETA + "/estudiantes.proto",  
                                         { keepCase: true });  
const paquete = grpc.loadPackageDefinition(definicion).estudiantes;  
module.exports = { grpc, paquete, definicion };
```

La opción keepCase: true conserva los nombres de campo tal como están escritos en el .proto. Sin ella, proto-loader los convertiría a estilo camelCase en JavaScript y usted vería en el código nombres que no existen en el contrato.

Paso 3. Mire lo que produjo la carga

Cree src/inspeccionar.js. No hace nada útil para la aplicación: sirve para ver qué construyó proto-loader a partir del contrato.

```
src/inspeccionar.js  
const { paquete, definicion } = require("../carga");  
  
console.log("Tipos y servicios declarados en el contrato:");  
console.log(Object.keys(definicion));  
  
console.log("\nMétodos de EstudianteService y su ruta:");  
for (const [nombre, m] of Object.entries(paquete.EstudianteService.service)) {  
  console.log("  ", nombre, "->", m.path);  
}
```

Ejecute node src/inspeccionar.js. Verá los mensajes y el servicio con su nombre completo, y la ruta de cada método: /estudiantes.EstudianteService/ObtenerEstudiante. Esa ruta es literalmente el camino HTTP/2 por el que viajará la llamada.

Captura N.º 3: salida de node src/inspeccionar.js con los tipos declarados y la ruta de los dos métodos. → *péguela en la Hoja de Evidencias.*

Paso 4. Compruebe qué es lo que viaja de verdad

Aquí está la idea central de la práctica. Cree `src/bytes.js`: serializa el mismo estudiante como protobuf y como JSON, y muestra los dos resultados.

```
src/bytes.js
const { definicion } = require("./carga");
const metodo = definicion["estudiantes.EstudianteService"].AgregarEstudiante;

const estudiante = { ci: "9876543", nombres: "Ana", apellidos: "Torres",
  carrera: "Sistemas" };

const binario = metodo.requestSerialize(estudiante);
const json = Buffer.from(JSON.stringify(estudiante));

console.log("protobuf:", binario.length, "bytes");
console.log(binario.toString("hex"));
console.log("JSON      :", json.length, "bytes");
console.log(json.toString());
```

Ejecute `node src/bytes.js`. En la línea hexadecimal busque el comienzo: `0a 07 39 38 37 36 35 34 33`. El primer byte, `0a`, es la etiqueta del campo: su valor se calcula como $(\text{número del campo} < 3) | \text{tipo}$, es decir $(1 < 3) | 2 = 10$, que en hexadecimal es `0a`. El `07` es la longitud y lo que sigue es «9876543». En ninguna parte aparece la palabra «ci».

Captura N.º 4: salida de `node src/bytes.js` mostrando los dos tamaños y el volcado hexadecimal del mensaje protobuf. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 2: En el volcado hexadecimal no aparece ninguno de los nombres de campo, solo sus números. Enuncie dos consecuencias prácticas de ese hecho para el equipo que mantiene el `.proto`: una favorable y una peligrosa. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Ya tiene el contrato y sabe qué viaja por el cable. Ahora hay que poner a alguien a atender del otro lado.

Laboratorio 2 — El servidor y la primera llamada unaria

Una llamada unaria es un mensaje de ida y un mensaje de vuelta: lo más parecido a una petición HTTP corriente. Es la forma que resuelve la enorme mayoría de las llamadas entre servicios, y es por donde se empieza.

Paso 1. Implemente los dos métodos del servicio

Cree `src/servidor.js`. Cada método recibe dos argumentos: la llamada, que trae el mensaje en su propiedad `request`, y una función para responder. Se responde una sola vez: o con un error, o con un resultado.

```
src/servidor.js
const { grpc, paquete } = require("./carga");
const estudiantes = new Map();

const servicio = {
  AgregarEstudiante: (llamada, responder) => {
    const e = llamada.request;
    if (!e.ci) return responder({ code: grpc.status.INVALID_ARGUMENT,
      message: "El campo ci es obligatorio" });
    estudiantes.set(e.ci, e);
    responder(null, { estudiante: e });
  },
  ObtenerEstudiante: (llamada, responder) => {
```

```
const e = estudiantes.get(llamada.request.ci);
if (!e) return responder({ code: grpc.status.NOT_FOUND,
                           message: "Estudiante no encontrado" });
responder(null, { estudiante: e });
},
};
```

El error no es una excepción: En gRPC el error se devuelve por el mismo camino que el resultado, con un código de estado. No lance excepciones dentro de un método: si lo hace, el cliente recibirá un genérico UNKNOWN (2) y perderá la información de qué salió mal.

Paso 2. Ponga el servidor a escuchar

Agregue al final del mismo archivo el bloque que registra el servicio y abre el puerto. La dirección se lee del entorno: en su máquina será localhost, y dentro de Docker tendrá que ser 0.0.0.0.

```
src/servidor.js (agregar al final)
const servidor = new grpc.Server();
servidor.addService(paquete.EstudianteService.service, servicio);

const DIR = process.env.GRPC_ADDR || "0.0.0.0:50051";
servidor.bindAsync(DIR, grpc.ServerCredentials.createInsecure(),
  (err, puerto) => {
    if (err) return console.error("No se pudo abrir el puerto:", err);
    console.log("Servicio gRPC escuchando en el puerto", puerto);
  });
```

Arranque con `node src/servidor.js` y deje esa terminal abierta durante todo el resto de la práctica. El 50051 es el puerto por convención de gRPC, igual que el 80 lo es de HTTP: no es una obligación, es un acuerdo.

Captura N.º 5: la terminal con el mensaje «Servicio gRPC escuchando en el puerto 50051». → *péguela en la Hoja de Evidencias.*

Paso 3. Llame al servicio sin escribir un cliente

Abra una SEGUNDA terminal en la carpeta del proyecto. `grpcurl` necesita el contrato para saber cómo armar el mensaje binario, y la opción `-plaintext` para no intentar una conexión cifrada contra un servidor que no la ofrece.

```
$ grpcurl -plaintext -proto proto/estudiantes.proto \
  -d '{"ci":"9876543","nombres":"Ana","apellidos":"Torres",
    "carrera":"Sistemas"}' \
  localhost:50051 estudiantes.EstudianteService/AgregarEstudiante

$ grpcurl -plaintext -proto proto/estudiantes.proto \
  -d '{"ci":"9876543"}' \
  localhost:50051 estudiantes.EstudianteService/ObtenerEstudiante
```

El argumento de `-d` va en una sola línea; aquí aparece partido solo para que quepa en la hoja. Las dos llamadas deben devolver el mismo estudiante dentro de un objeto «estudiante».

Captura N.º 6: las dos llamadas de `grpcurl` con sus respuestas en la segunda terminal. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 3: A la API REST de la Práctica 3 le pudo hablar con curl sin darle ningún archivo adicional; a este servicio hay que pasarle el .proto. Explique a qué se debe esa diferencia y qué gana y qué pierde un equipo por ella. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 3 — El cliente y los códigos de estado de gRPC

grpcurl sirve para probar, no para producir. El código que llama a un servicio interno es un cliente escrito en su mismo lenguaje, y lo que distingue a un cliente serio de uno de juguete es cómo trata los errores.

Paso 1. Escriba el cliente

Cree src/cliente.js. El cliente se construye a partir del mismo paquete que usó el servidor: por eso los dos leen el .proto.

```
src/cliente.js
const { grpc, paquete } = require("./carga");

const DIR = process.env.GRPC_ADDR || "localhost:50051";
const cliente = new paquete.EstudianteService(DIR,
  grpc.credentials.createInsecure());

const ana = { ci: "9876543", nombres: "Ana", apellidos: "Torres",
  carrera: "Sistemas" };

cliente.AgregarEstudiante(ana, (err, res) => {
  if (err) return console.error("Falló:", err.code, err.details);
  console.log("Agregado:", res.estudiante);
  cliente.ObtenerEstudiante({ ci: ana.ci }, (err2, res2) => {
    if (err2) return console.error("Falló:", err2.code, err2.details);
    console.log("Obtenido:", res2.estudiante);
  });
});
```

Ejecute node src/cliente.js con el servidor corriendo en la otra terminal. Observe que las llamadas se encadenan dentro de la respuesta anterior: gRPC en Node.js es asíncrono, igual que cualquier llamada de red.

Captura N.º 7: salida de node src/cliente.js con el estudiante agregado y luego obtenido. → *péguela en la Hoja de Evidencias.*

Paso 2. Pida un estudiante que no existe

Cree src/buscar.js: recibe la cédula por la línea de comandos y, cuando falla, imprime el código numérico y su nombre.

```
src/buscar.js
const { grpc, paquete } = require("./carga");

const cliente = new paquete.EstudianteService(
  process.env.GRPC_ADDR || "localhost:50051",
  grpc.credentials.createInsecure());

cliente.ObtenerEstudiante({ ci: process.argv[2] || "0000000" },
  (err, res) => {
    if (err) {
      console.error("código :", err.code, "=", grpc.status[err.code]);
      console.error("detalle:", err.details);
      return;
    }
    console.log("Encontrado:", res.estudiante);
  });
```

Ejecute `node src/buscar.js 0000000` y después `node src/buscar.js 9876543`. El primero debe imprimir «código : 5 = NOT_FOUND» y el detalle que escribió el servidor; el segundo, el estudiante.

Captura N.º 8: las dos ejecuciones de `buscar.js`: la que devuelve NOT_FOUND con su código 5 y la que encuentra al estudiante. → *péguela en la Hoja de Evidencias.*

Paso 3. Compare los códigos de gRPC con los de HTTP

gRPC no usa los códigos de estado de HTTP: define los suyos, dieciséis en total, y viajan en una cabecera al final de la llamada. La correspondencia con HTTP es aproximada y conviene tenerla a la vista.

referencia · equivalencias entre gRPC y HTTP		
gRPC	HTTP	cuándo aparece
OK (0)	200	la llamada terminó bien
INVALID_ARGUMENT (3)	400	el mensaje llegó, los datos no sirven
DEADLINE_EXCEEDED (4)	504	el cliente dejó de esperar la respuesta
NOT_FOUND (5)	404	lo que se pidió no existe
ALREADY_EXISTS (6)	409	el estado actual impide la operación
PERMISSION_DENIED (7)	403	identificado, pero sin autorización
UNIMPLEMENTED (12)	501	el método no existe en ese servicio
INTERNAL (13)	500	falla del servidor, no del que llama
UNAVAILABLE (14)	503	no se pudo hablar con el servidor

Observe la diferencia más útil de todas: DEADLINE_EXCEEDED y UNAVAILABLE describen dos fallas distintas que en HTTP suelen llegar mezcladas. La primera dice «el servidor quizá esté trabajando todavía»; la segunda, «nunca llegué a hablar con él». Reintentar es seguro en un caso y peligroso en el otro.

Pregunta N.º 4: Su servicio A llama a su servicio B y recibe un error. Explique por qué la decisión de reintentar la llamada no puede ser la misma si el código es UNAVAILABLE (14) que si es DEADLINE_EXCEEDED (4). Use un ejemplo de su proyecto integrador en el que reintentar por error causaría un daño real. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Provoque UNIMPLEMENTED y DEADLINE_EXCEEDED

Los dos errores que más aparecen en esta práctica conviene verlos ahora, provocados a propósito, y no dentro de tres laboratorios sin saber de dónde salen.

```
# un método que el servicio no implementa: UNIMPLEMENTED (12)
$ grpcurl -plaintext -proto proto/estudiantes.proto \
  -d '{"ci":"9876543"}' \
    localhost:50051 estudiantes.EstudianteService/BorrarEstudiante

# un plazo de un milisegundo: el cliente deja de esperar
$ grpcurl -plaintext -max-time 0.001 -proto proto/estudiantes.proto \
  -d '{"ci":"9876543"}' \
    localhost:50051 estudiantes.EstudianteService/ObtenerEstudiante
```

El primero falla porque el método está en el `.proto` de `grpcurl` pero no en el servicio... salvo que tampoco esté en el `.proto`, en cuyo caso `grpcurl` ni siquiera intenta llamar. Pruebe las dos variantes y anote la diferencia: una es un error del contrato y la otra un error del servidor.

Captura N.º 9: las dos llamadas fallidas mostrando los códigos `Unimplemented` y `DeadlineExceeded`. → *péguela en la Hoja de Evidencias.*

Con esto ya sabe hacer una llamada, recibir una respuesta y distinguir cinco maneras de fracasar. Falta la forma de llamada que REST no tiene.

Laboratorio 4 — Streaming: cuando una sola respuesta no alcanza

Hasta aquí todo fue unario: pido, espero, recibo. gRPC admite además tres formas con flujo, y la más útil para empezar es la del servidor: el cliente hace una petición y el servidor devuelve una secuencia de mensajes por la misma conexión, sin esperar a tenerlos todos.

Paso 1. Declare la llamada de flujo en el contrato

Primero el contrato, siempre. Agregue el mensaje de petición y el nuevo rpc a `proto/estudiantes.proto`. La palabra `stream` delante del mensaje de salida es todo el cambio.

```
proto/estudiantes.proto (agregar)
message ListarRequest {
  string carrera = 1; // vacío significa: todas las carreras
}

service EstudianteService {
  rpc AgregarEstudiante (Estudiante) returns (EstudianteResponse);
  rpc ObtenerEstudiante (EstudianteRequest) returns (EstudianteResponse);
  rpc ListarEstudiantes (ListarRequest) returns (stream Estudiante);
}
```

Reemplace el bloque `service` completo por este: el rpc nuevo se agrega a los dos que ya tenía. Recuerde que el servicio debe reiniciarse para que lea el contrato modificado.

Paso 2. Implemente el flujo en el servidor

Un método con flujo de salida recibe un solo argumento: la llamada. En lugar de responder una vez, escribe tantos mensajes como haga falta y después cierra.

```
src/servidor.js (agregar dentro de servicio)
ListarEstudiantes: (llamada) => {
  const carrera = llamada.request.carrera;
  let enviados = 0;
  for (const e of estudiantes.values()) {
    if (carrera && e.carrera !== carrera) continue;
    llamada.write(e);
    enviados++;
  }
  console.log("Enviados", enviados, "estudiantes por el flujo");
  llamada.end();
},
```

write y end: `llamada.write()` envía un mensaje; `llamada.end()` cierra el flujo con estado OK. Si olvida el `end()`, el cliente se queda esperando para siempre y usted verá un `DEADLINE_EXCEEDED` que parece un problema de red y no lo es.

Paso 3. Consuma el flujo desde el cliente

Cree `src/listar.js`. Un flujo en Node.js se consume con eventos: uno por cada mensaje que llega, uno al terminar y uno si algo falla. Además mide en qué milisegundo llegó cada elemento.

```
src/listar.js
const { grpc, paquete } = require("../carga");

const cliente = new paquete.EstudianteService(
  process.env.GRPC_ADDR || "localhost:50051",
```

```
grpc.credentials.createInsecure());

const inicio = process.hrtime.bigint();
const flujo = cliente.ListarEstudiantes({ carrera: process.argv[2] || "" });

flujo.on("data", (e) => {
  const ms = Number(process.hrtime.bigint() - inicio) / 1e6;
  console.log(ms.toFixed(1), "ms ->", e.ci, e.nombres, e.carrera);
});
flujo.on("end", () => console.log("fin del flujo"));
flujo.on("error", (err) => console.error(err.code, err.details));
```

Reinicie el servidor, vuelva a agregar a Ana con node src/cliente.js y ejecute node src/listar.js. Con un solo estudiante el resultado es poco impresionante: lo interesante viene en el paso siguiente.

Captura N.º 10: salida de node src/listar.js mostrando el estudiante recibido por el flujo y el mensaje «fin del flujo». → *péguela en la Hoja de Evidencias.*

Paso 4. Cargue muchos datos y mire cuándo llega el primero

Cree src/sembrar.js para llenar el servicio con quinientos estudiantes y volver a listar. Ahí se ve la diferencia entre una respuesta única y un flujo.

```
src/sembrar.js
const { grpc, paquete } = require("./carga");

const cliente = new paquete.EstudianteService(
  process.env.GRPC_ADDR || "localhost:50051",
  grpc.credentials.createInsecure());

const carreras = ["Sistemas", "Redes", "Industrial"];
let pendientes = 500;

for (let i = 1; i <= 500; i++) {
  const e = { ci: String(1000000 + i), nombres: "Estudiante " + i,
    apellidos: "Prueba", carrera: carreras[i % 3] };
  cliente.AgregarEstudiante(e, (err) => {
    if (err) return console.error(err.code, err.details);
    if (--pendientes === 0) console.log("Sembrados 500 estudiantes");
  });
}
```

Ejecute node src/sembrar.js y después node src/listar.js Sistemas. Fíjese en el milisegundo del primer elemento y en el del último: el cliente pudo empezar a trabajar con el primero mucho antes de que terminara la respuesta. Con una llamada unaria que devolviera un arreglo, el primer dato utilizable habría llegado recién al final.

Captura N.º 11: salida de node src/listar.js Sistemas con los tiempos del primer y del último estudiante recibidos. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 5: Describa una operación de su proyecto integrador en la que el flujo del servidor sería una mejora real frente a devolver el arreglo completo, y otra en la que sería una complicación sin beneficio. Justifique cada caso con el volumen de datos y con lo que el cliente hace mientras espera. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 5 — Evolución del contrato: agregar sin romper, y romper a propósito

Un contrato que no puede cambiar está muerto. Todo el diseño de protobuf gira alrededor de una sola pregunta: cómo agregar campos a un mensaje mientras hay clientes ejecutándose con la versión anterior, sin coordinar un despliegue simultáneo. Esta es, con diferencia, la parte de la práctica que más se parece a la vida real.

Paso 1. Congele el contrato actual como versión 1

Antes de tocar nada, guarde una copia del contrato tal como está. Esa copia representa al «cliente viejo»: el que ya está desplegado y que nadie va a actualizar hoy.

```
$ cp -r proto proto-v1
$ ls proto-v1
# en PowerShell: Copy-Item -Recurse proto proto-v1
# proto-v1 no se vuelve a tocar en toda la práctica
```

Gracias a la variable `PROTO_DIR` que puso en `src/carga.js`, cualquiera de sus clientes puede ejecutarse con el contrato viejo sin cambiar una sola línea de código.

Paso 2. Agregue un campo nuevo con un número nuevo

El servicio necesita ahora el correo institucional del estudiante. Modifique el mensaje `Estudiante` en `proto/estudiantes.proto` —solo el de la carpeta `proto`, nunca el de `proto-v1`—.

```
proto/estudiantes.proto (modificar el mensaje Estudiante)
message Estudiante {
  string ci          = 1;
  string nombres     = 2;
  string apellidos   = 3;
  string carrera     = 4;
  string correo      = 5; // campo nuevo, con un número nunca usado
}
```

Reinicie el servidor y agregue un estudiante con correo usando `grpcurl` o modificando `src/cliente.js`. El servidor ya conoce el campo 5; el cliente viejo, no.

Paso 3. Compruebe que el cliente viejo sigue funcionando

Ejecute la misma consulta dos veces: primero con el contrato nuevo y después con el congelado.

```
# cliente NUEVO: conoce el campo correo
$ node src/buscar.js 9876543

# cliente VIEJO: usa el contrato sin el campo correo
$ PROTO_DIR=./proto-v1 node src/buscar.js 9876543

# en PowerShell:
$ $env:PROTO_DIR="./proto-v1"; node src/buscar.js 9876543
```

El cliente viejo responde igual que siempre, sin errores: simplemente no muestra el correo porque su contrato no declara el campo 5, y protobuf descarta en silencio lo que no reconoce. Eso es compatibilidad hacia adelante, y es la razón por la que se puede desplegar un servicio nuevo sin coordinar con todos sus consumidores.

Captura N.º 12: las dos ejecuciones seguidas: el cliente nuevo mostrando el correo y el cliente viejo respondiendo bien sin ese campo. → *péguela en la Hoja de Evidencias.*

Paso 4. Rompa el contrato reutilizando un número

Ahora provoque el error que esta práctica quiere que usted no cometa nunca en producción. Suponga que alguien decide reordenar los campos «para que queden más prolijos» y reutiliza números ya asignados.

```
proto/estudiantes.proto (cambio deliberadamente incorrecto)
message Estudiante {
  string ci           = 1;
  string nombres      = 2;
  string apellidos    = 3;
  string correo       = 4;    // MAL: el 4 era carrera
  string carrera      = 5;    // MAL: el 5 era correo
}
```

Reinicie el servidor, vuelva a agregar al estudiante y consulte otra vez con los dos clientes. El cliente viejo pide el campo 4 esperando la carrera y recibe el correo: imprime un dato equivocado en un campo con nombre correcto, sin un solo error, sin una excepción, sin una línea en el registro. Nadie se entera hasta que alguien lee un informe y ve una dirección de correo en la columna «Carrera».

Captura N.º 13: la salida del cliente viejo mostrando el valor equivocado en el campo carrera, sin ningún mensaje de error. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: Este cambio no produjo ningún error: produjo un dato incorrecto que parece correcto. Compárelo con lo que habría pasado en la API REST de la Práctica 3 si alguien renombra un campo del JSON. ¿Cuál de las dos fallas es más fácil de detectar y por qué? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 5. Prevenga la ruptura con reserved

protobuf ofrece la manera de que ese error sea imposible: declarar como reservados los números y los nombres retirados. Corrija el mensaje dejándolo así.

```
proto/estudiantes.proto (versión correcta y definitiva)
message Estudiante {
  reserved 4;
  reserved "carrera";
  string ci           = 1;
  string nombres      = 2;
  string apellidos    = 3;
  string correo       = 5;
  string carrera_actual = 6;    // el reemplazo lleva número propio
}
```

Con reserved, cualquier intento futuro de volver a usar el número 4 o el nombre «carrera» falla al cargar el contrato, antes de que el servicio arranque. Vuelva a poner el 4 en algún campo y compruébelo: el error aparece en el arranque, no en producción seis meses después.

Captura N.º 14: el error de carga del .proto al intentar reutilizar un número reservado, y el arranque correcto una vez corregido. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 7: Formule, en una sola frase y con sus palabras, la regla que su equipo debería escribir en el README del repositorio para que nadie vuelva a reutilizar el número de un campo. Indique además quién debería revisarla y en qué momento del trabajo. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Con esto ya sabe agregar sin romper y, más importante, sabe cómo se rompe. Solo queda comprobar si todo este trabajo vale la pena, y eso no se opina: se mide.

Laboratorio 6 — Medir contra REST y empaquetar en la red de Docker

La afirmación «gRPC es más rápido que REST» no vale nada sin un número al lado. Vamos a pedir el MISMO dato de las dos maneras —al servicio REST de la Práctica 3 y al servicio gRPC de hoy— y a medir dos cosas: cuántos bytes viajan y cuánto tarda. Después empaquetaremos el servicio gRPC y haremos que el REST lo consuma, que es el lugar que le corresponde en la arquitectura: hacia afuera REST, hacia adentro gRPC.

Paso 1. Mida el camino REST

Con el servicio de la Práctica 3 corriendo, mida el tamaño y el tiempo de una consulta por identificador.

```
$ curl -s -o /dev/null \
  -w "%{size_download} bytes   %{time_total} s\n" \
  http://localhost:3000/usuarios/1

# repita la medición diez veces seguidas y quédese con la mediana,
# nunca con el mejor tiempo: la primera llamada siempre miente
```

Anote el tamaño del cuerpo y el tiempo. Ese tamaño es JSON: nombres de campo, comillas, llaves y espacios incluidos.

Captura N.º 15: las mediciones con curl del servicio REST, con el tamaño en bytes y el tiempo de cada llamada. → *péguela en la Hoja de Evidencias.*

Paso 2. Mida el camino gRPC

Cree src/medir.js. Hace diez llamadas unarias seguidas, promedia el tiempo y muestra el tamaño de la respuesta serializada en protobuf junto al de la misma respuesta en JSON.

```
src/medir.js
const { grpc, paquete, definicion } = require("./carga");
const cliente = new paquete.EstudianteService(
  process.env.GRPC_ADDR || "localhost:50051",
  grpc.credentials.createInsecure());
const svc = definicion["estudiantes.EstudianteService"];
let n = 0, suma = 0;
(function medir() {
  const t0 = process.hrtime.bigint();
  cliente.ObtenerEstudiante({ ci: "9876543" }, (err, res) => {
    if (err) return console.error(err.code, err.details);
    suma += Number(process.hrtime.bigint() - t0) / 1e6;
    if (++n < 10) return medir();
    console.log("promedio de 10 llamadas:", (suma / n).toFixed(2), "ms");
    console.log("protobuf:", svc.ObtenerEstudiante.responseSerialize(res).length,
      "bytes · JSON:", Buffer.byteLength(JSON.stringify(res)));
  });
})();
```

Ejecute node src/medir.js. Compare los dos tamaños que imprime: es el mismo dato, contado de dos maneras. Y compare el promedio con el tiempo de curl, recordando que aquí no hay arranque de proceso en cada llamada.

Captura N.º 16: salida de node src/medir.js con el promedio de las diez llamadas y los dos tamaños de la respuesta. → *péguela en la Hoja de Evidencias.*

Paso 3. Llene la tabla de comparación de los tres estilos

Cree informe/comparacion.md con SUS números. No se evalúa qué estilo gana: se evalúa que los números sean suyos y que la conclusión se siga de ellos.

informe/comparacion.md

Medida	REST (P3)	GraphQL (P4)	gRPC (P5)
Bytes de la respuesta			
Tiempo promedio observado (ms)			
Viajes de red del cliente			
Formato que viaja	texto	texto	binario
¿Se puede leer sin el contrato?	sí	sí	no
¿El navegador lo consume directo?	sí	sí	no
Público al que sirve mejor			

La última fila es la importante y la debe llenar usted con una sola palabra por columna. Si en las tres escribe lo mismo, vuelva a mirar las filas anteriores.

Captura N.º 17: la tabla de comparación completada con sus propias mediciones de las Prácticas 3, 4 y 5.
→ *péguela en la Hoja de Evidencias.*

Pregunta N.º 8: Con sus tres columnas a la vista: ¿por qué sería un error exponer este servicio gRPC directamente al navegador de un usuario final, aunque sea el más rápido de los tres? Y a la inversa: ¿por qué sería un error que sus servicios internos se hablen entre ellos por REST si ya tienen este contrato? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Empaquete el servicio gRPC

Cree el Dockerfile en la raíz de practica5-grpc. Preste atención a la línea que copia la carpeta proto: en Node.js el contrato se lee en tiempo de ejecución, así que si no viaja en la imagen, el contenedor no arranca.

```
Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY proto ./proto
COPY src ./src
ENV GRPC_ADDR=0.0.0.0:50051
EXPOSE 50051
CMD ["node", "src/servidor.js"]
```

Por qué 0.0.0.0 y no localhost: Dentro de un contenedor, localhost es el propio contenedor. Un servidor que escucha en 127.0.0.1 es invisible para los demás contenedores de la red, aunque el proceso esté perfectamente vivo. Es el error más frecuente de este laboratorio.

Paso 5. Haga que el servicio REST lo consuma

En el proyecto de la Práctica 3, cree el cliente gRPC. Copie allí la carpeta proto/ —los dos extremos necesitan el mismo contrato— y agregue este archivo.

```
practica3-rest/src/estudiantes.cliente.js
const grpc = require("@grpc/grpc-js");
const protoLoader = require("@grpc/proto-loader");

const RUTA = __dirname + "../proto/estudiantes.proto";
const def = protoLoader.loadSync(RUTA, { keepCase: true });
const paquete = grpc.loadPackageDefinition(def).estudiantes;
const DIR = process.env.GRPC_ADDR || "localhost:50051";
const cliente = new paquete.EstudianteService(DIR,
    grpc.credentials.createInsecure());
```

```
function obtenerEstudiante(ci) {
  return new Promise((ok, mal) => {
    cliente.ObtenerEstudiante({ ci }, (err, res) => {
      err ? mal(err) : ok(res.estudiante);
    });
  });
}

module.exports = { obtenerEstudiante };
```

Agregue en el servicio REST una ruta GET /usuarios/:id/expediente que llame a obtenerEstudiante y devuelva el resultado en JSON. Si la llamada gRPC falla con NOT_FOUND (5), responda 404; si falla con UNAVAILABLE (14), responda 503. Ahí se ve, en una sola función, la traducción entre los dos mundos: gRPC hacia adentro, HTTP hacia afuera.

Paso 6. Levante los dos servicios en la misma red

Cree docker-compose.yml en practica5-grpc. Observe que el servicio gRPC NO publica puertos hacia su máquina: solo se lo alcanza desde dentro de la red de Docker, que es exactamente lo que se espera de un servicio interno.

```
docker-compose.yml
services:
  estudiantes:
    build: .
    # sin sección ports: no se expone fuera de la red de Docker

  usuarios:
    build: ../practica3-rest
    ports:
      - "3000:3000"
    environment:
      GRPC_ADDR: estudiantes:50051
    depends_on:
      - estudiantes
```

Levante todo con `docker compose up --build` y pruebe desde su máquina con `curl -i http://localhost:3000/usuarios/1/expediente`. El nombre «estudiantes» de la variable GRPC_ADDR no es una dirección IP ni un alias que usted haya configurado: es el nombre del servicio en el compose, y lo resuelve el DNS interno de Docker. Compruebe también que `curl` a `localhost:50051` no responde: ese puerto no está publicado.

Captura N.º 18: la respuesta de `curl` al servicio REST devolviendo datos que vinieron por gRPC, junto a la salida de `docker compose ps` con los dos contenedores. → *péguela en la Hoja de Evidencias.*

Paso 7. Suba el trabajo al repositorio

Incluya un README.md que explique el orden para levantar todo y qué contiene cada carpeta, en especial por qué existe `proto-v1`.

```
$ git init && git add .
$ git commit -m "Práctica 5: servicio gRPC interno consumido por el REST"
$ git remote add origin <URL-de-su-repositorio>
$ git push -u origin main
```

Captura N.º 19: la página del repositorio en GitHub mostrando la carpeta `practica5-grpc` con `proto/`, `proto-v1/`, `src/`, el `Dockerfile` y el `README`. → *péguela en la Hoja de Evidencias.*

Cierre la sesión con esta idea: REST para el público, GraphQL para las pantallas y gRPC para la conversación interna. No son tres opciones entre las que hay que elegir una, sino tres capas que en un sistema real conviven, y usted acaba de construir las tres.

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Todos los ejercicios se resuelven sobre SU proyecto integrador, no sobre el servicio de estudiantes del laboratorio. Del 1 al 3 son la base; el 4 y el 5 son los que separan una entrega correcta de una sobresaliente, y ninguno de los dos se puede aprobar sin evidencia.

Ejercicio 1. El contrato .proto de un servicio interno de su proyecto (10 pts)

Elija dos servicios de su proyecto integrador que se comuniquen entre sí y escriba el contrato .proto de esa comunicación. No es un ejercicio de traducción mecánica de sus rutas REST: es un ejercicio de diseño del contrato.

Se pide:

- Un archivo .proto con al menos tres mensajes y un servicio con cuatro operaciones, con la numeración de campos comentada donde no sea obvia.
- Al menos un mensaje de petición propio en lugar de un tipo simple, con la justificación de por qué se previó así el crecimiento.
- Un campo enum o repeated usado con criterio, explicando qué decisión de diseño representa.
- La salida de un script como el src/inspeccionar.js del Laboratorio 1 sobre SU contrato, demostrando que carga sin errores.

Criterio de aceptación: *Un compañero debe poder leer solo el .proto y decir qué hace el servicio, sin ver una línea de su implementación.*

Ejercicio 2. Servidor y cliente con los códigos de estado correctos (12 pts)

Implemente el servidor de ese contrato y un cliente que lo consuma. La mayor parte del puntaje está en el tratamiento de los errores, no en el camino feliz.

Se pide:

- Las cuatro operaciones implementadas, con datos que sobrevivan mientras el proceso viva.
- Al menos cuatro códigos de estado distintos devueltos por el servidor en las situaciones que les corresponden: como mínimo NOT_FOUND, INVALID_ARGUMENT y ALREADY_EXISTS.
- Un cliente que distinga los códigos y actúe distinto ante cada uno, con la captura de las cuatro respuestas de error.
- Una tabla en el informe que relacione cada código que usted devuelve con el código HTTP que la puerta pública debería exponer en su lugar.

Criterio de aceptación: *Ningún error de negocio puede llegar al cliente como OK con un campo «error» dentro del mensaje. Si eso ocurre, el ejercicio no acredita.*

Ejercicio 3. Una llamada con flujo del servidor sobre datos reales (10 pts)

Agregue a su servicio una operación con flujo del servidor sobre una colección de su proyecto que tenga, al menos, quinientos elementos.

Se pide:

- El rpc con stream declarado en el contrato y su implementación en el servidor.

- Un cliente que consuma el flujo con eventos y registre el instante en que llegó cada mensaje.
- Una medición del tiempo hasta el primer elemento y hasta el último, comparada con la misma consulta devuelta como una sola respuesta.
- Una explicación de qué hace el cliente con el primer elemento mientras todavía están llegando los demás. Si la respuesta es «nada, espera a tenerlos todos», el flujo no estaba justificado y así debe decirlo.

Criterio de aceptación: Las dos mediciones deben existir y ser suyas. La conclusión puede ser que el flujo no aportaba: eso puntúa igual que lo contrario, si está medido.

Ejercicio 4. Evolución del contrato sin romper a los clientes viejos (10 pts)

Demuestre sobre su propio servicio las dos caras de la evolución de un contrato: el cambio compatible y el cambio que corrompe datos en silencio.

Se pide:

- Una copia congelada de su contrato como versión 1, y un cliente capaz de ejecutarse con cualquiera de las dos versiones.
- Un campo nuevo agregado con un número nunca usado, con la evidencia de que el cliente viejo sigue funcionando sin cambios.
- Una ruptura provocada reutilizando el número de un campo existente, con la captura del dato incorrecto que recibe el cliente viejo sin ningún mensaje de error.
- La corrección con reserved sobre el número y el nombre retirados, y la evidencia de que un intento posterior de reutilizarlos falla al cargar el contrato.
- Tres reglas escritas en el README de su repositorio para que su equipo no repita esa ruptura.

Criterio de aceptación: La captura de la corrupción silenciosa es obligatoria: sin ella el ejercicio no acredita, porque es la única evidencia de que entendió el problema y no solo la solución.

Ejercicio 5. La misma operación medida en REST y en gRPC (10 pts)

Tome una operación que su proyecto ya resuelve por REST desde la Práctica 3 y resuélvala también por gRPC. Después mida las dos, no opine sobre las dos.

Se pide:

- La misma operación implementada de las dos maneras, devolviendo exactamente los mismos datos.
- Al menos diez repeticiones de cada medición, informando la mediana y no el mejor tiempo, con el método descrito en el informe.
- Los dos tamaños de carga en bytes, del cuerpo REST y del mensaje protobuf, y el porcentaje de diferencia.
- Una tabla comparativa que incluya la columna de GraphQL de la Práctica 4, aunque sea con las mediciones que ya tomó entonces.
- Una conclusión que responda a una pregunta concreta: en qué punto de su arquitectura cambiaría REST por gRPC, y en qué punto no lo haría ni aunque fuera más rápido.

Criterio de aceptación: Toda afirmación de la conclusión debe poder señalarse en una celda de su tabla. Una conclusión sin mediciones que la respalden no acredita el ejercicio.

Ejercicio 6. El servicio interno dentro de la red de Docker (8 pts)

Empaquete su servicio gRPC y haga que su servicio REST lo consuma por nombre de servicio, sin publicar el puerto de gRPC hacia el exterior.

Se pide:

- Dockerfile del servicio gRPC que incluya el contrato en la imagen y escuche en 0.0.0.0.
- Un docker-compose.yml donde el servicio gRPC no publique puertos y el REST lo alcance por su nombre de servicio.

- Una ruta del servicio REST que devuelva datos obtenidos por gRPC, con la traducción de al menos dos códigos de estado gRPC a sus equivalentes HTTP.
- La evidencia de que el puerto 50051 NO responde desde su máquina y de que la ruta REST sí funciona.
- El README con el orden de arranque y una explicación de qué resuelve el nombre «estudiantes» en la variable de entorno.

Criterio de aceptación: Se probará clonando su repositorio y ejecutando `docker compose up --build`. Si hace falta un paso manual no documentado, el ejercicio no acredita.

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 19 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 8 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. El contrato .proto de un servicio interno de su proyecto	Contrato legible y completo, mensajes de petición propios, numeración deliberada, carga sin errores demostrada.	10
2. Servidor y cliente con los códigos de estado correctos	Cuatro operaciones funcionando, cuatro códigos correctos y justificados, cliente que los distingue, traducción a HTTP razonada.	12
3. Una llamada con flujo del servidor sobre datos reales	Flujo declarado e implementado, cliente por eventos, medición del primer y último elemento, uso del flujo justificado con honestidad.	10
4. Evolución del contrato sin romper a los clientes viejos	Versión congelada y cliente doble, campo agregado sin ruptura, corrupción silenciosa demostrada, reserved aplicado y verificado, reglas escritas.	10
5. La misma operación medida en REST y en gRPC	Misma operación en los dos estilos, método de medición descrito, diez repeticiones, tamaños y tiempos, conclusión sostenida por los números.	10
6. El servicio interno dentro de la red de Docker	Imagen que incluye el contrato, puerto interno no publicado, resolución por nombre de servicio, traducción de códigos, arranque reproducible.	8
Subtotal		60

Penalizaciones

Situación	Descuento
Reutilizar el número de un campo retirado sin declararlo reserved en la entrega final	-6 pts
Errores de negocio devueltos como OK con un campo «error» dentro del mensaje, sin usar los códigos de estado de gRPC	-5 pts
Servidor que escucha en 127.0.0.1 dentro del contenedor, o puerto 50051 publicado hacia el exterior sin justificación	-4 pts
Conclusiones del ejercicio 5 sin la tabla de mediciones que las respalde	-5 pts
Reescribir el servicio REST de la Práctica 3 en lugar de agregarle el cliente gRPC	-4 pts

Situación	Descuento
node_modules subido al repositorio (falta .gitignore)	-3 pts
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
12 UNIMPLEMENTED: The server does not implement the method	El método está declarado en el .proto pero no en el objeto de implementación, o el nombre no coincide exactamente.	Compare el nombre del rpc con la clave del objeto servicio: distinguen mayúsculas. Reinicie el servidor tras editar el contrato.
4 DEADLINE_EXCEEDED: Deadline exceeded	El plazo del cliente venció, o un método con flujo nunca llamó a llamada.end() y el cliente quedó esperando.	Verifique que todo camino del método termine en responder() o en end(). Amplíe el plazo solo después de descartar eso.
Error: ENOENT: no such file or directory, open ".../estudiantes.proto"	La ruta al contrato es relativa al proceso y no al archivo, o el .proto no se copió a la imagen de Docker.	Arme la ruta con __dirname y agregue COPY proto ./proto en el Dockerfile.
Error: illegal token / expected ",'" (line N)	Sintaxis del .proto: falta un punto y coma, un tipo mal escrito o un import colocado después de su uso.	Los import van siempre al comienzo del archivo. Lea la línea que indica el error, no la anterior.
Un campo llega vacío o con el valor de otro campo	El número del campo cambió entre el contrato del cliente y el del servidor. Lo que viaja es el número, no el nombre.	Compare los dos .proto campo por campo. Nunca reutilice un número: declárelo reserved y use uno nuevo.
14 UNAVAILABLE: No connection established	El servidor no está levantado, escucha en otra dirección, o el cliente apunta a un puerto equivocado.	Confirme el mensaje de arranque del servidor y la variable GRPC_ADDR del cliente antes de tocar cualquier otra cosa.
14 UNAVAILABLE con «Handshake failed» o «Connection dropped»	Las credenciales no coinciden: cliente con createInsecure contra servidor con TLS, o grpcurl sin la opción -plaintext.	En laboratorio, use createInsecure en los dos extremos y -plaintext en grpcurl. En producción, TLS en los dos.
TypeError: Cannot read properties of undefined (reading "service")	El nombre del paquete usado en JavaScript no coincide con la línea package del .proto.	Si el contrato dice package estudiantes;, el acceso es loadPackageDefinition(def).estudiantes y nada más.

Antes de pedir ayuda: Antes de pedir ayuda, responda tres preguntas: ¿qué código de estado devolvió la llamada y qué significa según la tabla del Laboratorio 3?, ¿el .proto que carga el cliente es exactamente el mismo que carga el servidor?, y ¿el servidor imprimió su mensaje de arranque? Esos tres datos resuelven la mayoría de los casos sin ayuda de nadie.

Referencias

- Documentación oficial de gRPC para Node.js — <https://grpc.io/docs/languages/node/>
- Guía del lenguaje Protocol Buffers, proto3 — <https://protobuf.dev/programming-guides/proto3/>
- Reglas de actualización de un contrato sin romper clientes — <https://protobuf.dev/programming-guides/proto3/#updating>
- grpcurl, cliente de línea de comandos — <https://github.com/fullstorydev/grpcurl>
- Newman, Sam. Building Microservices, 2.ª edición, capítulo sobre estilos de comunicación. O'Reilly, 2021.
- Diapositivas del Tema 4 — Comunicación entre Microservicios (COM-600).